

IGNITE

Vehicle Inventory and Sales Management System for SMEs

FINAL PROJECT REPORT

University of Technology and Entrepreneurship

Department of Computer Science

Advanced OOP with Python | Class CSM1

Term 5 | Year 2

Prepared by	Lecturer
Lay Katsrieng Sour Chansokpanha Heang Sovannara	Sek Socheat

Report prepared 24 September 2026
Implementation reviewed at commit a2f79b8
Branch refactor/clean-structure

Abstract and report guide

IGNITE is a web application for managing individual vehicles, customers, inventory movements, sales and invoices in a small or medium-sized dealership. The project addresses the risk of inconsistent records when stock and sales information are maintained separately. It combines a browser interface with Python service objects, Flask routes, repositories and a MySQL relational database.

The central workflow completes a sale within one database transaction. It verifies the active operator and customer, locks the available vehicle, records the sale, changes its status to SOLD, records a stock-out movement and creates an invoice. Role-based access control separates administrative, management and sales duties. Vehicle photo validation, decimal money calculations and database constraints provide additional safeguards.

A fresh execution of the existing regression suite passed all 51 tests with no failures or skips. The tests cover normal workflows, rejected input, concurrent sale attempts, rollback, permissions, persistence, seed safety and image handling. Screenshots in this report were captured from an isolated database using the project seed data. They demonstrate implemented screens rather than customer deployment results.

The implementation meets the proposal's core inventory and sales objectives. It remains a university development system: online payments, accounting, multi-branch synchronization and legal tax invoicing are outside the current scope. No field study or measured business productivity improvement is claimed.

Contents

Section	Coverage	Page
1 Introduction	Background, problem, objectives and scope	3
2 Technology Review	Flask, MySQL, OOP and related systems	4
3 Analysis and Design	Requirements, use cases, architecture, classes and ERD	6
4 Implementation	Packages, key decisions, code and screenshots	13
5 Testing and Results	Execution evidence, test cases, corrections and security	20
6 Conclusion	Achievements, limitations, lessons and future work	23
Appendices A to D	SQL, user manual, Git evidence and team log	24
References	Proposal, lecturer requirements, code and official documentation	31

Evidence basis: supplied proposal and lecturer screenshot; current source, schema and tests; read-only Git history; fresh test run and isolated browser captures on 24 September 2026. Individual contribution details require team confirmation.

1 Introduction

1.1 Background

A dealership handles distinct physical assets. Two vehicles can share a brand, model and year but still require separate records because their VINs, prices, condition and sales histories differ. Inventory therefore needs to identify each vehicle and connect it to the people and transactions that affect its availability.

The proposal identifies manual records, spreadsheets and separate files as possible sources of delay and inconsistency for vehicle-selling SMEs [1]. IGNITE responds with a centralized application in which authorized staff can search vehicle records, maintain customer information and follow a sale through to its invoice. This motivation comes from the project proposal; it is not presented as the result of an external dealership survey.

1.2 Problem statement

When inventory and sales are updated independently, staff may see an outdated availability status, duplicate a sale or lose the connection between a vehicle, customer and invoice. Uncontrolled editing can also expose administrative functions to users who do not need them. The technical problem is to maintain these relationships and enforce business rules consistently across browser requests and database writes.

1.3 Objectives

Objective from the proposal	Implemented response
Manage individual vehicle records	Create, list, filter, edit and view vehicles; preserve historical records through deactivation.
Track inventory movements	Record STOCK_IN, STOCK_OUT and ADJUSTMENT against a vehicle and operator.
Process sales and invoices	Complete sale, update availability, record stock-out and issue invoice atomically.
Control user access	Authenticate active users and authorize actions using role permissions.
Apply layered OOP design	Separate dataclasses, services, repositories, routes and database connection ownership.

1.4 Scope and stakeholders

The main operators are Admin, Manager and Sales Staff. An SME owner can use an authorized account to review business information, while customers receive invoice information without a customer login portal. The system covers one dealership workspace and uses USD in its interface. Online payment, GPS tracking, a native mobile application, external registration checks, AI pricing, multi-branch synchronization and full accounting remain excluded, consistent with the proposal [1].

2 Technology Review

2.1 Python and Flask

Python supports the project's domain records, business services, validation and tests. Flask provides routing, request handling and an explicit application object; its factory pattern supports constructing separate application instances for testing [3]. IGNITE implements `create_app` in `app/__init__.py` and keeps the local launch entry point in `app.py`. Jinja renders server-side HTML, while CSS and vanilla JavaScript implement the interface.

This stack keeps the project runnable without a separate frontend build process. It also makes the separation between request handling and business operations visible for an Advanced OOP course. The application uses direct MySQL queries through repositories rather than an object-relational mapper. Consequently, transaction boundaries and SQL mapping are explicit responsibilities of the project code.

2.2 MySQL and persistent relationships

MySQL stores ten tables using InnoDB. Foreign keys connect vehicles, customers, operators, sales and invoices; junction tables implement the many-to-many access relationships. Unique constraints prevent duplicate vehicle codes and duplicate sales of the same vehicle. CHECK constraints restrict status values and monetary relationships.

A normal read is not sufficient when two requests may sell the same vehicle. MySQL locking reads using `SELECT ... FOR UPDATE` lock selected rows within a transaction [4]. IGNITE combines these locks with a unique `sales.vehicle_id` constraint and rollback handling. The database layer uses `READ COMMITTED` isolation for service transactions. These mechanisms are verified by the concurrent-sale and rollback tests, rather than assumed from the interface alone.

2.3 Verified development environment

Component	Observed version or use
Python	3.14.6 in the project virtual environment
Flask	3.1.3
MySQL Connector/Python	9.7.0
MySQL server	8.0.46; tests require actual MySQL 8
Pillow	12.3.0 for validating and re-encoding photos
python-dotenv	1.2.3 for local configuration
Rendering and interface	Jinja templates, responsive CSS and vanilla JavaScript
Testing and version control	unittest and Git; refactor/clean-structure branch

These are the observed versions used for this report, not a promise that every future installation will resolve to the same packages. `requirements.txt` specifies supported ranges. Credentials and the session secret are supplied through environment configuration and are omitted from this report.

2 Technology Review continued

2.4 Object-oriented programming in IGNITE

The implementation uses eight domain dataclasses: User, Role, Permission, Vehicle, Customer, Sale, StockMovement and Invoice. Dataclasses generate record methods from declared fields [5]. Most of these objects represent data rather than implementing all business behavior themselves. Services own the workflows, which is an important distinction from the broader model responsibilities described in the proposal.

OOP concept	Concrete project example	Reason
Encapsulation	Database.transaction; VehiclePhotoService	Centralizes connection ownership and file lifecycle rules.
Abstraction	MySQLRepository exposes get, save, transaction and queries	Services describe business operations without repeating SQL.
Inheritance	VehicleRepository and other entity repositories extend EntityRepository	Reuses projection, lookup, persistence and mapping behavior.
Polymorphism	Common repository operations use each subclass's fields, model and table	A shared interface works with different entity definitions.
Composition	SalesService receives a repository; VehicleService receives a photo service	Collaborating objects keep responsibilities separate.
Value object	Frozen SaleAmounts with calculate and total	Keeps validated decimal price and discount together.

The repository mapper converts dataclass records back to dictionaries so existing templates and route contracts remain compatible. This choice preserves the interface while making domain fields explicit. It is not a fully domain-driven model: many validation rules are intentionally located in services and form helpers.

2.5 Related approaches

A spreadsheet can list vehicle attributes, but relationships, concurrent sales and permissions would need additional conventions or automation. IGNITE implements these rules centrally. A general inventory system offers a broader comparison: Odoo documents unique serial-number tracking and traceability for individual products [6]. IGNITE applies a narrower concept to vehicles by keeping one row per physical vehicle and associating its movement and sale history.

This comparison concerns design scope, not measured performance or product superiority. IGNITE does not implement the breadth of warehouse, procurement or accounting functions found in a larger enterprise system. Its educational value lies in a complete, inspectable vehicle sale workflow and a clear separation of Python responsibilities.

3 Analysis and Design

3.1 Functional and nonfunctional requirements

ID	Requirement	Verification basis
FR01	Authenticate active users and reject unauthorized actions.	Login, CSRF and server permission tests.
FR02	Maintain searchable, filterable vehicle records and optional photos.	Vehicle forms, pagination and photo tests.
FR03	Maintain customer details and retain customer history.	Customer forms and detail routes.
FR04	Record legal stock transitions for a physical vehicle.	Inventory transition tests.
FR05	Allow a sale only for an available vehicle and active participants.	Sale success, invalid-sale and concurrency tests.
FR06	Create one invoice and stock-out entry with each completed sale.	Atomic sale and forced-failure tests.
FR07	Present totals, status counts and date-filtered sales information.	Report/decimal tests and browser screenshots.
FR08	Manage accounts, role assignment and role permissions.	Role, inactive-user and last-admin tests.
NFR01	Preserve data across application instances.	Fresh-app persistence integration test.
NFR02	Keep related sale writes consistent under errors and concurrency.	Rollback and duplicate-sale tests.
NFR03	Separate presentation, rules and persistence for maintainability.	Package and source inspection.
NFR04	Handle invalid input and database outages predictably.	Validation and friendly 503 tests.

3.2 Seed role permissions

Function	Admin	Manager	Sales Staff
View vehicles, customers, sales, invoices	Yes	Yes	Yes
Create/update customers and complete sales	Yes	Yes	Yes
Create/update vehicles; manage inventory	Yes	Yes	No
View reports	Yes	Yes	No
Vehicle deactivation permission	Yes	No	No
Manage users and role permissions	Yes	No	No

This matrix describes database/seed.sql defaults. Administrators can edit permitted role grants, so a working database may differ. The schema supports multiple roles per user, but the current account editor assigns one role. The proposed Assistant role is future work.

3 Use cases

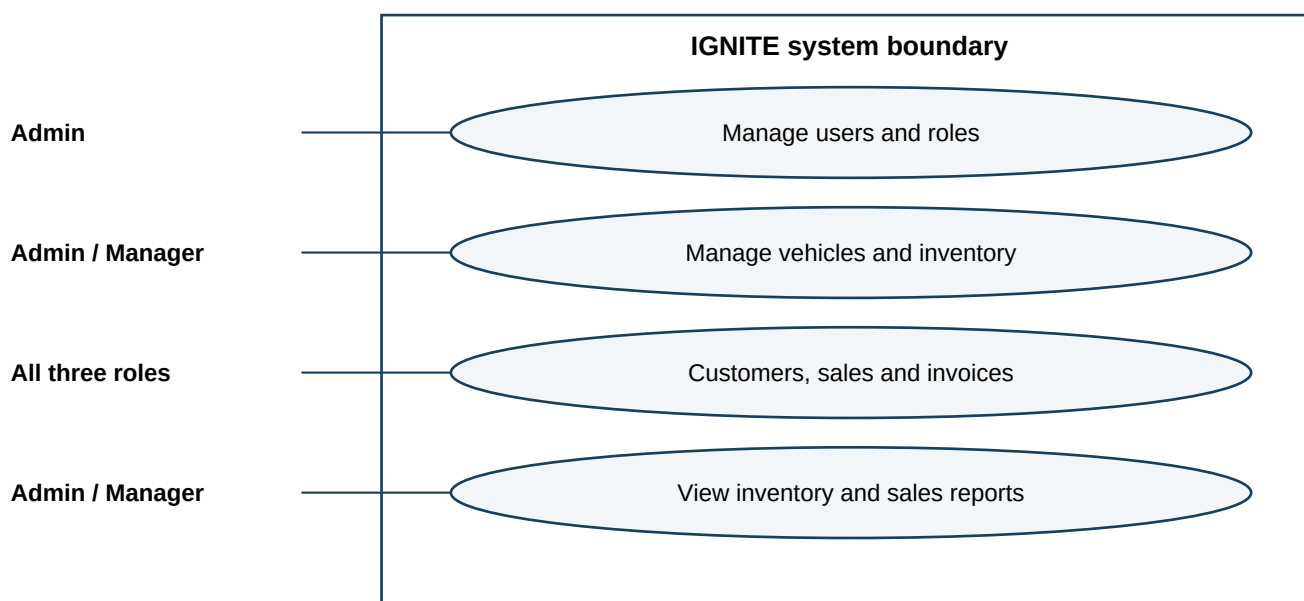


Figure 3.1. Main actor associations under the default seeded permissions. Every use case requires an authenticated account and the relevant server-side permission.

3.3 Complete a vehicle sale

Element	Description
Primary actor	Admin, Manager or Sales Staff with the sales permissions.
Preconditions	Active operator; active customer; vehicle status AVAILABLE; valid discount.
Normal flow	Open New Sale, select the customer and available vehicle, review price and discount, then submit completion. The service locks records and commits sale, SOLD status, stock-out and invoice together.
Postconditions	A completed sale exists, its vehicle is SOLD, one stock-out entry is recorded and one invoice is linked by sale_id.
Alternate flow	If another request sold the vehicle, or an input is invalid, reject the sale. If invoice insertion fails, roll back all changes.
Boundary	The current workflow completes immediately. It does not implement deposits, instalments, cancellation or refunds.

3.4 Record an inventory movement

An authorized operator selects a vehicle, chooses a movement and supplies a reason. STOCK_IN requires INACTIVE and changes it to AVAILABLE. Manual STOCK_OUT changes an in-stock vehicle to INACTIVE. ADJUSTMENT records a zero-quantity note without changing status. SOLD vehicles are rejected by this workflow; a sale has its own stock-out operation.

3.5 Maintain user access

An administrator creates or edits an account and assigns an active role. A new password is hashed before persistence. The service rejects changes that would remove the last active administrator and prevents the current operator from changing their own access through the account editor. Administrative grants are protected from normal role-permission editing.

3 Layered architecture

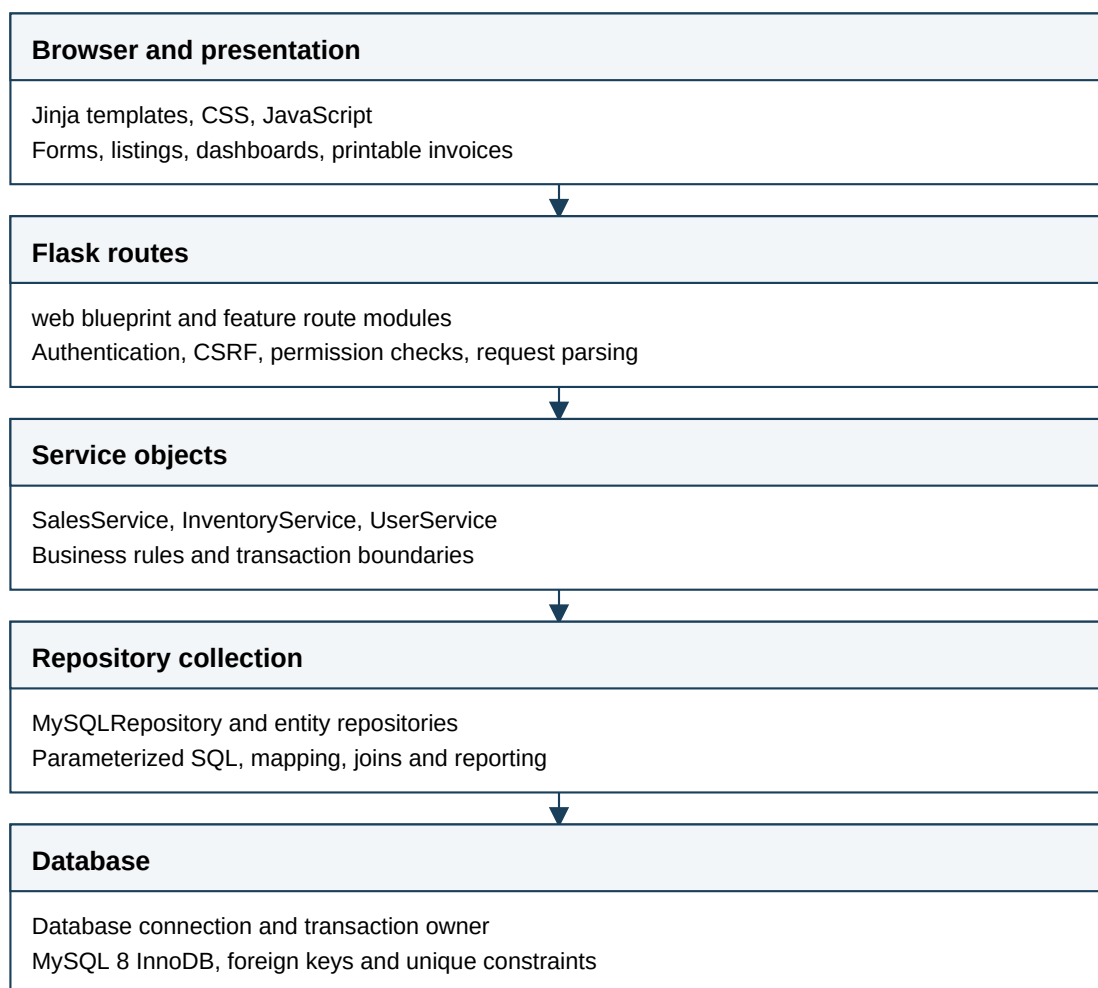


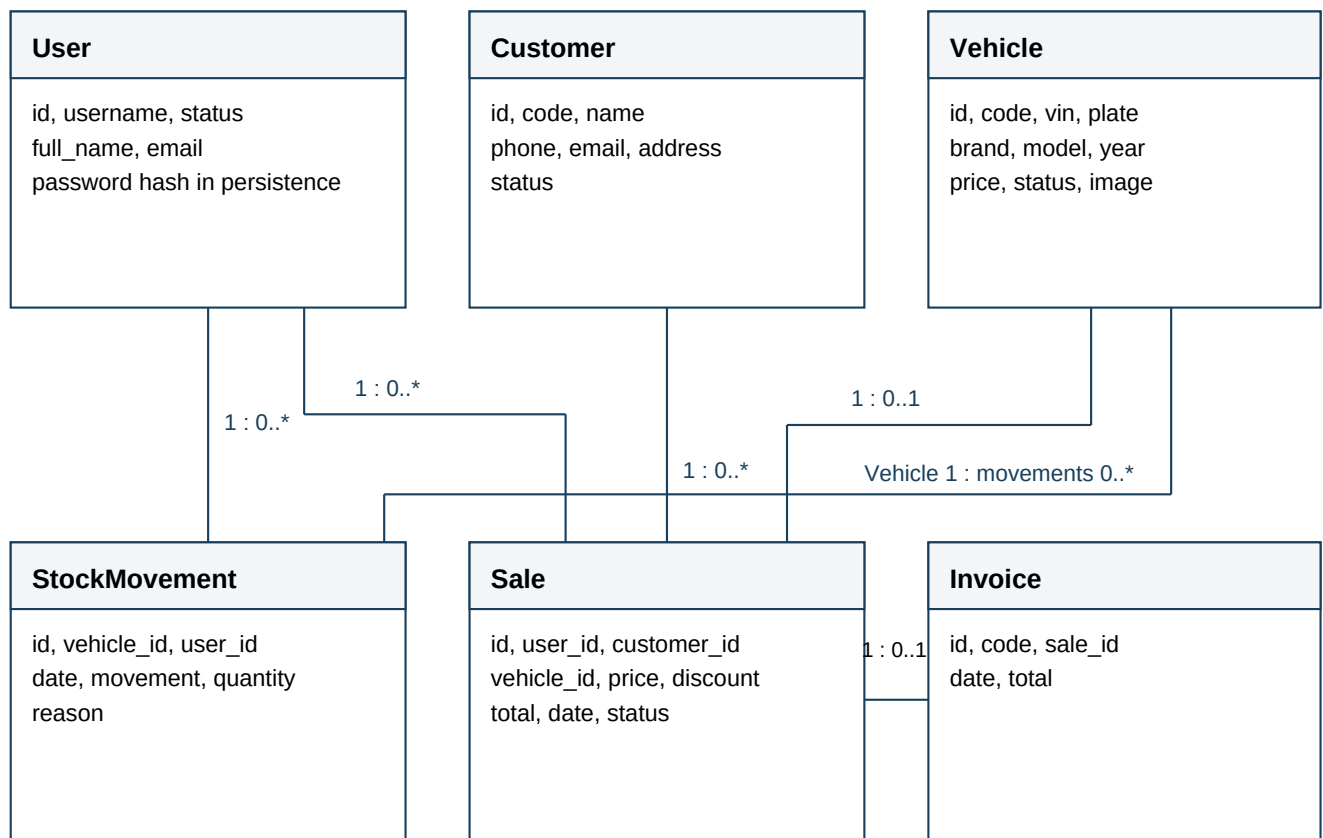
Figure 3.2. Main write workflow. Read routes may request repository data directly or through ReportService; domain records support mapping and money validation across layers.

3.6 Responsibility boundaries

The presentation layer renders the outcome; it does not decide whether a vehicle can be sold. Routes parse requests, check access and call a service. SalesService and InventoryService coordinate business operations. Entity repositories own table-specific SQL, while MySQLRepository provides a shared collection and joined views. Database owns the connection and transaction context used by all participating repositories.

This arrangement prevents a service from accidentally committing only one part of a sale. It also allows tests to construct a new application with an isolated database. Request teardown closes the request connection, and database exceptions are translated into application-level errors instead of exposing raw SQL or credentials to the user.

3 Class design



User * : * Role via user_roles; Role * : * Permission via role_permissions

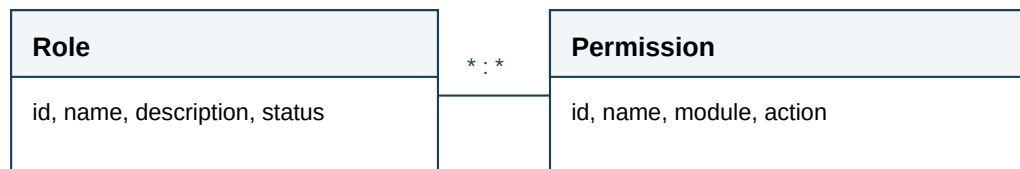


Figure 3.3. Domain record associations. Multiplicities show relational possibilities; a successful completed-sale workflow creates exactly one invoice. Attributes are selected for readability rather than listing every timestamp or joined display field.

3.7 Key class behavior

Class	Implemented responsibility
SaleAmounts	calculate validates price and discount; total returns their difference using Decimal.
SalesService	complete coordinates all writes for a completed sale.
InventoryService	record validates physical-vehicle transitions and their signed quantities.
UserService	save manages accounts; permissions updates editable role grants.
VehiclePhotoService	Validates, stores, replaces and safely cleans up uploaded images.
EntityRepository	Provides select, get, save, exists and dataclass mapping.

3 Operational database design

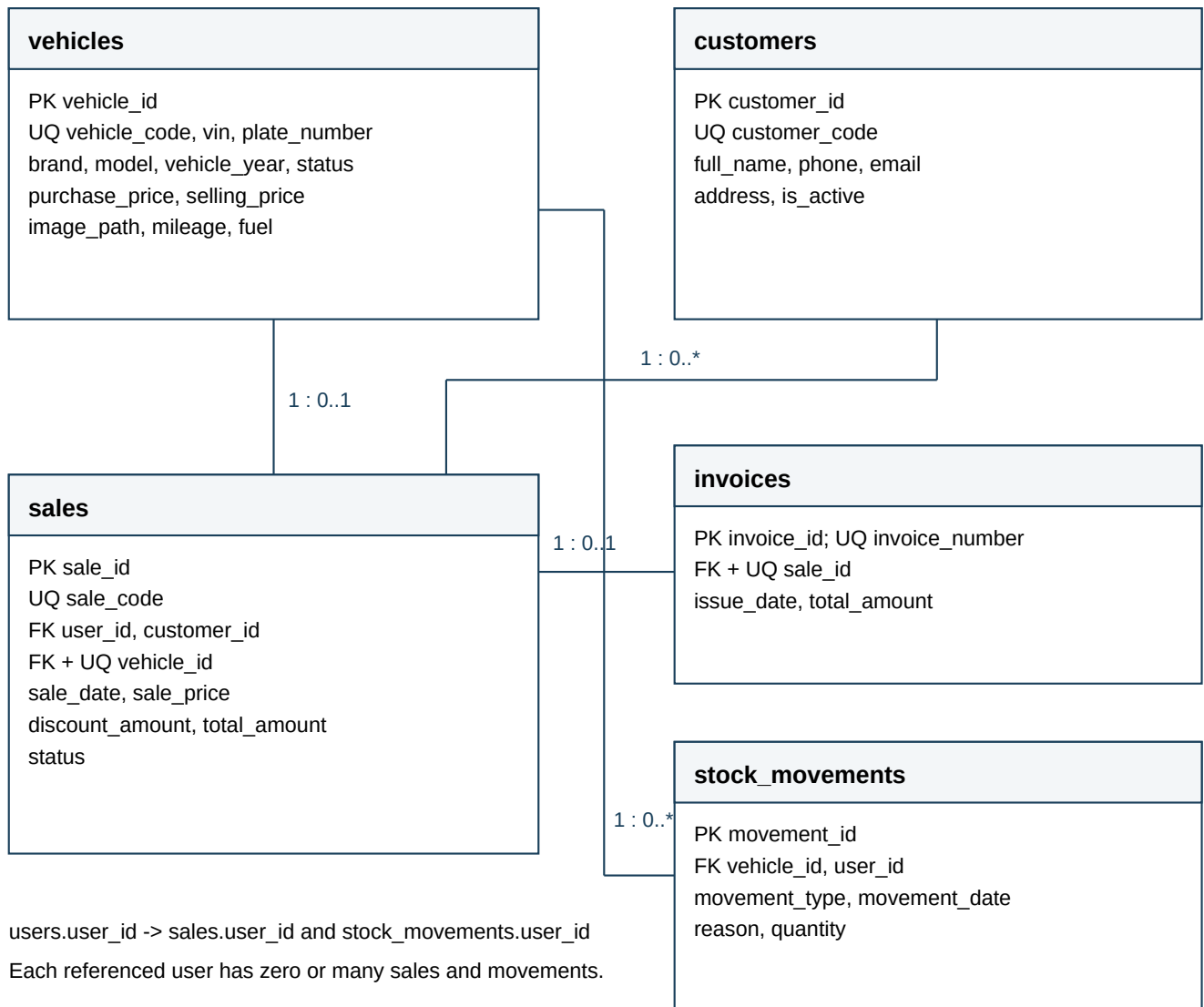


Figure 3.4. Operational ERD derived from database/schema.sql. PK = primary key; FK = foreign key; UQ = unique. Role tables are shown separately in Figure 3.5.

3.8 Integrity and historical records

A vehicle may have zero or one sale because sales.vehicle_id is unique. A sale may have at most one stored invoice because invoices.sale_id is unique. The service adds the stronger workflow guarantee that a successfully completed sale receives an invoice. Customers and operators can be linked to many sales, and each vehicle can have many stock movements.

Stock movements do not contain a sale_id foreign key. A sale-generated movement is associated with its vehicle and operator; its reason includes the sale code. This preserves the implemented schema rather than inventing a direct Sale-to-StockMovement relationship. Foreign keys retain identity when a readable code changes. Historical records are preserved through deactivation rather than ordinary hard deletion.

3 Access control database design

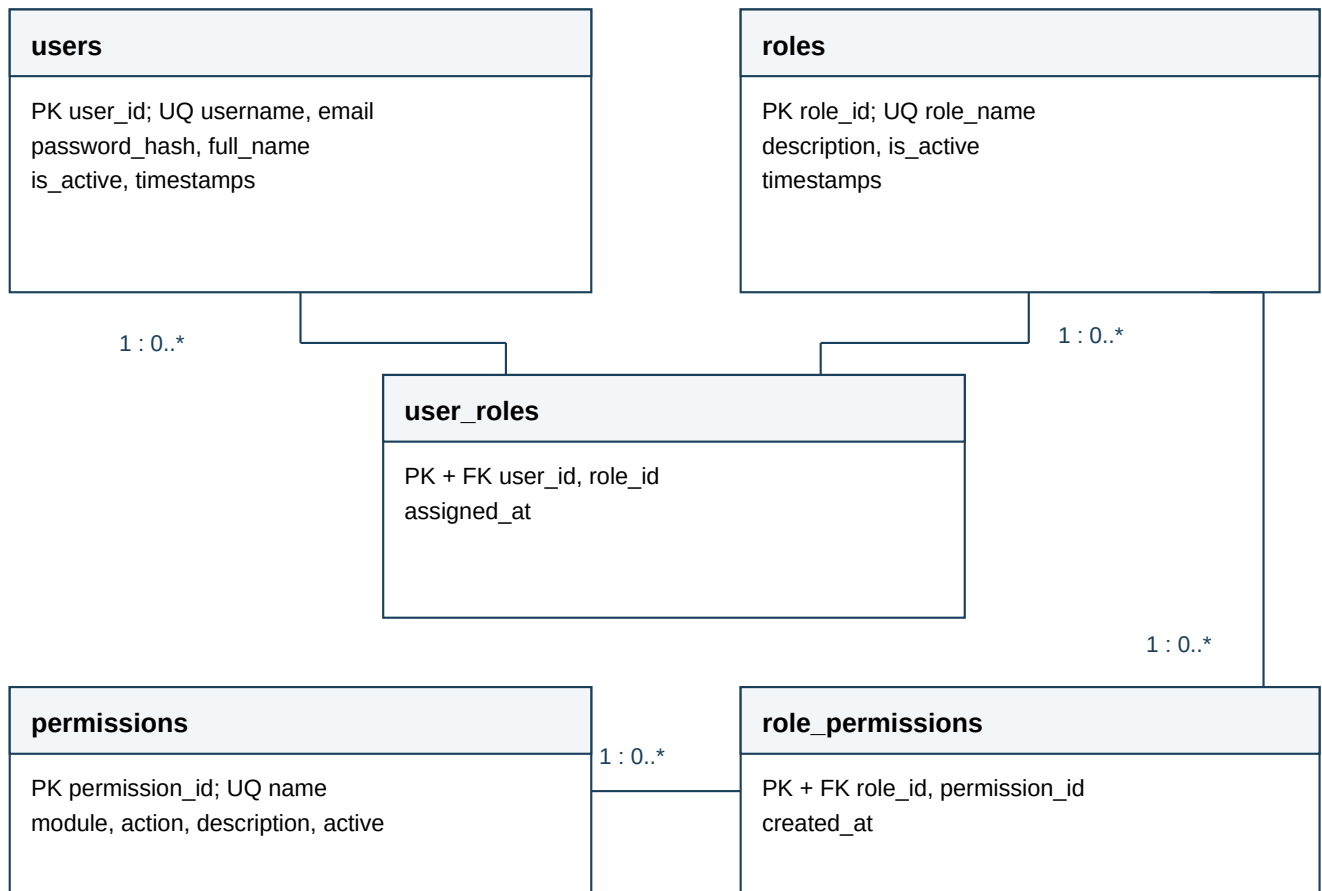


Figure 3.5. Access-control ERD. Composite keys prevent duplicate user-role and role-permission assignments. Permission name denotes permission_name in the physical schema.

3.9 Data design choices

Numeric primary keys carry relationships; readable codes are unique labels. Money uses DECIMAL(14,2). VIN and plate values have uniqueness constraints, and non-null VINs require 17 characters. Vehicle states are AVAILABLE, RESERVED, SOLD and INACTIVE. Sale states permit PENDING, COMPLETED and CANCELLED, but the implemented workflow completes sales immediately; schema options do not establish a cancellation process.

3.10 Proposal and implementation alignment

Proposal element	Current implementation
Ten relational tables	All ten are present; actual data types and constraints are reproduced in Appendix A.
InvoiceService	Invoice creation is included in the atomic SalesService workflow.
Domain validation methods	Most rules are in services/form helpers; SaleAmounts supplies domain money behavior.
UserRole and RolePermission classes	Represented as composite-key junction tables and repository operations.
Assistant role consideration	Not seeded or implemented as a dedicated role.
Delete or deactivate records	Historical records are retained through deactivation.

3 Transaction and state rules

3.11 Atomic completion sequence

Step	Operation	Purpose
1	Begin a shared transaction at READ COMMITTED.	All participating repositories use one connection.
2	Lock operator, customer and vehicle with FOR UPDATE.	Validate current active/available state under concurrent requests.
3	Calculate SaleAmounts from stored price and discount.	Reject invalid money and excessive discount.
4	Insert sale with a temporary unique code, then assign its ID-based code.	Avoid count-based identifier races.
5	Set vehicle SOLD and insert STOCK_OUT quantity -1.	Record both current state and movement history.
6	Insert invoice referencing the sale primary key.	Maintain one invoice for the transaction.
7	Commit; on any exception roll back.	Prevent partial sales, orphan invoice links or incorrect stock state.

3.12 Inventory transitions

Current state	Operation	Result
INACTIVE	STOCK_IN, quantity +1	AVAILABLE
AVAILABLE or RESERVED	Manual STOCK_OUT, quantity -1	INACTIVE
Any non-SOLD state	ADJUSTMENT, quantity 0	State unchanged; reason recorded
AVAILABLE	Complete sale, STOCK_OUT -1	SOLD, completed sale and invoice
SOLD	Manual inventory operation	Rejected

Vehicle registration establishes the initial state. The system does not derive availability solely by summing stock quantities: an adjustment is an audit note, and initial registration is not equivalent to every later stock-in action. The distinction between vehicle state and movement history is necessary when interpreting reports.

3.13 Identifier conventions

Clean seed data uses VEH-001 through VEH-020, SALE-000001 through SALE-000008 and matching INV-000001 through INV-000008. Fictional 17-character VIN-like values, such as 9ZZ1K8R2000000013, are unique testing identifiers rather than certified or registered VINs. The current live sale generator uses SAL- plus a six-digit ID; this differs from the seed's SALE- prefix and is recorded here as an existing consistency limitation.

4 Implementation

4.1 Package structure

vehicle_sales_system/	
app.py	Local server entry point
config.py	Environment and upload settings
app/	
__init__.py	Application factory
database.py	Connection and transaction owner
models/	Eight records and money rules
repositories/	
base.py	Shared entity repository behavior
mysql.py	Repository collection and joined views
*_repository.py	Entity queries and reporting
services/	Business workflows and photo lifecycle
routes/	
__init__.py	Register the web blueprint
*_routes.py	Feature request handlers
shared_routes.py	Common list/form/detail dispatch
common.py	Access checks, CSRF and navigation
form_helpers.py	Parsing and validation
resource_config.py	Resource fields and metadata
templates/	Jinja page templates
static/	Styles, scripts and bundled assets
database/	Schema, seed, setup and migrations
tests/	Regression and integration tests
docs/	Implementation and verification notes

The refactor separates previously combined code by responsibility while preserving the existing web endpoint names and template data contracts. Each entity repository declares its table, fields and model. The shared base creates projections and maps SQL rows through the declared dataclass. Joined details and reporting queries remain in the repository layer.

4.2 Entry point and configuration

app.py creates the Flask application and launches a local server on 127.0.0.1:5000 with debug disabled. Configuration loads DB_HOST, DB_PORT, DB_NAME, DB_USER, DB_PASSWORD and SECRET_KEY from the environment. A missing session secret is rejected. Startup does not seed or reset the database; explicit database/setup.py commands manage initialization.

4.3 Request handling

Feature route modules handle authentication, dashboard, vehicles, customers, inventory, sales, invoices, reports, users and roles. Common resource routes reduce repeated listing and form code. Validation errors preserve submitted form values. A resource identifier comes from server-controlled metadata; SQL values use query parameters instead of concatenating untrusted input into a statement.

4 Key implementation decisions

4.4 Decimal money as a value object

```
@dataclass(frozen=True)
class SaleAmounts:
    price: Decimal
    discount: Decimal

    @classmethod
    def calculate(cls, price, discount):
        result = cls(money(price), money(discount))
        if result.discount > result.price:
            raise ValueError(
                "Discount must be between zero and the selling price.")
        return result

    @property
    def total(self):
        return self.price - self.discount
```

Listing 4.1. SaleAmounts from app/models/sale.py, with line wrapping adjusted for the report. The money helper rejects non-finite, negative and excessive amounts and quantizes values to two decimal places. The database independently checks the price-discount relationship.

4.5 Shared transaction ownership

Database.transaction starts a transaction, publishes its connection through a ContextVar, commits on success and rolls back on exceptions. Nested service transactions are rejected. This makes SalesService the owner of a whole sale rather than allowing each repository write to commit independently. The repository get operation adds FOR UPDATE only when the caller requests a lock.

4.6 Photo lifecycle

VehiclePhotoService accepts JPEG, PNG and WebP uploads up to 5 MB and 20 million pixels. It validates decoded content against the extension, rejects animation, applies orientation, resizes within 1920 by 1920 pixels and re-encodes without metadata. UUID-based filenames prevent the original client filename from choosing a storage path.

The database stores a relative image path. On replacement, the previous file is deleted only after the database commit and only if no vehicle still references it. Failed writes preserve the previous photo and clean up the new file where safe. Bundled illustrations and paths outside the upload directory are protected from cleanup. A process crash can still leave an orphan file, so database and filesystem operations are not claimed to be one atomic storage transaction.

4.7 Seed and compatibility decisions

Editable sample definitions are centralized in database/seed.py; database/seed.sql contains role and permission reference data. Identifier migration previews its exact changes and checks collisions before application. It changes labels while preserving primary keys and foreign keys. Old internal preference keys, test prefixes and compatibility names are retained intentionally; visible project branding is IGNITE.

4 Login and dashboard screens

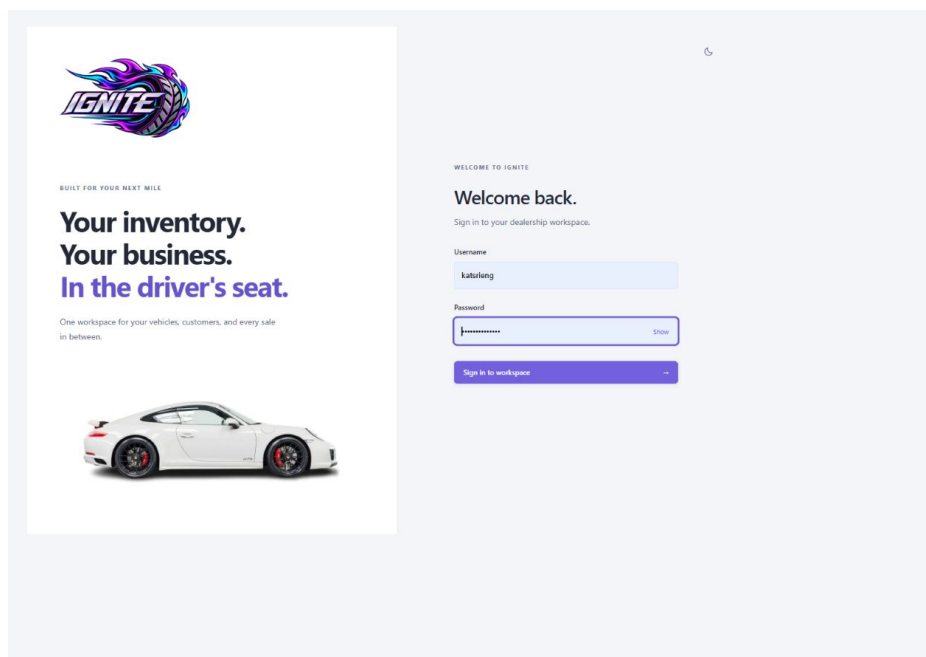


Figure 4.1. Current login page with IGNITE branding and the supplied white sports-car illustration. No password is displayed.

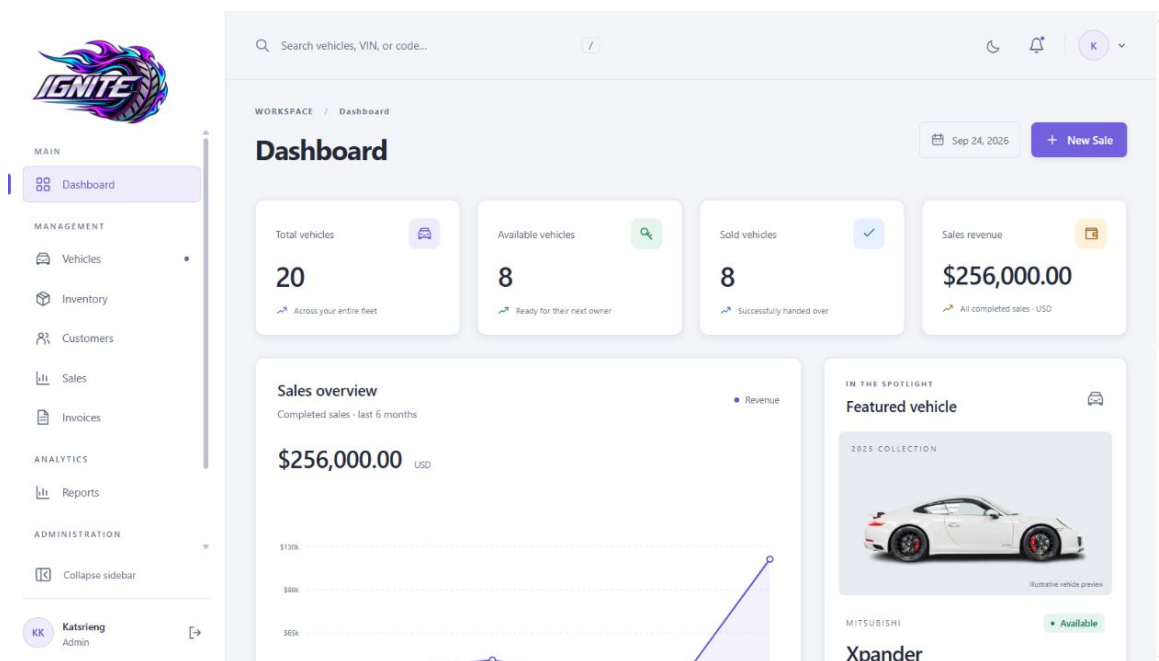


Figure 4.2. Dashboard from the isolated seeded database: 20 vehicles, 8 available, 8 sold and USD 256,000 completed-sale revenue.

The dashboard provides counts, a revenue chart and a featured vehicle. The car artwork is a generic illustration, not evidence of the listed vehicle's actual appearance. All screenshots in Figures 4.1 to 4.9 were captured on 24 September 2026 from the current source at the reviewed commit.

4 Vehicles and inventory screens

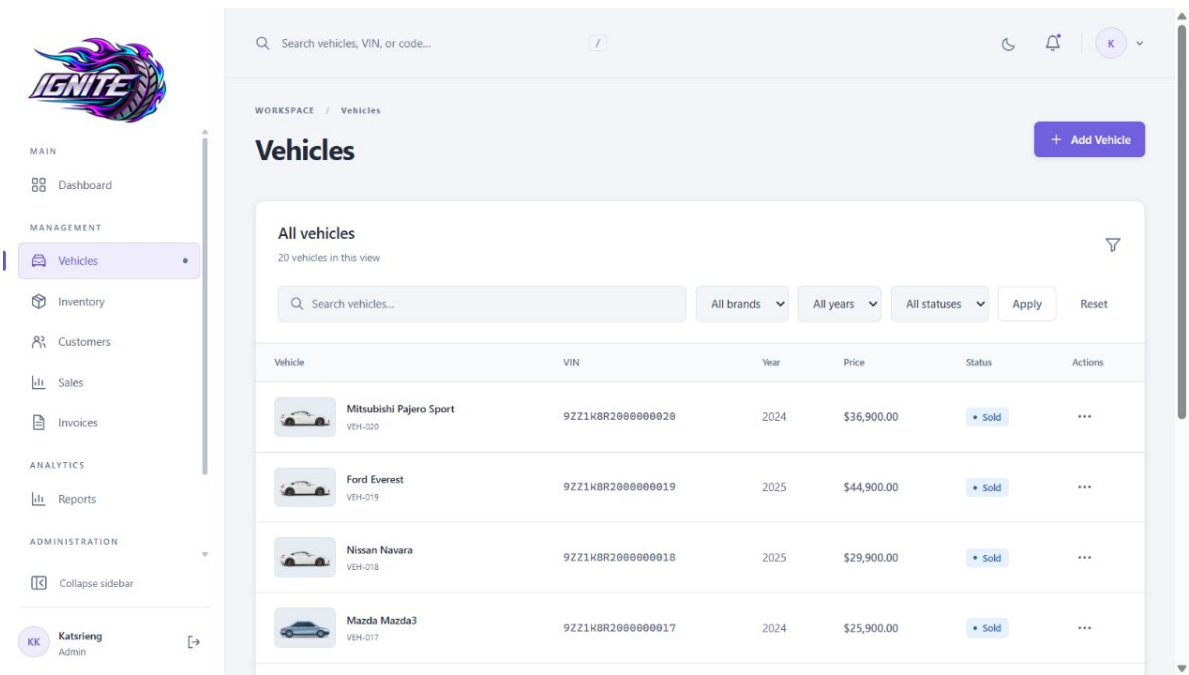


Figure 4.3. Vehicle listing with unique codes and VIN-like values, status filters and pagination. Each row represents one physical vehicle.

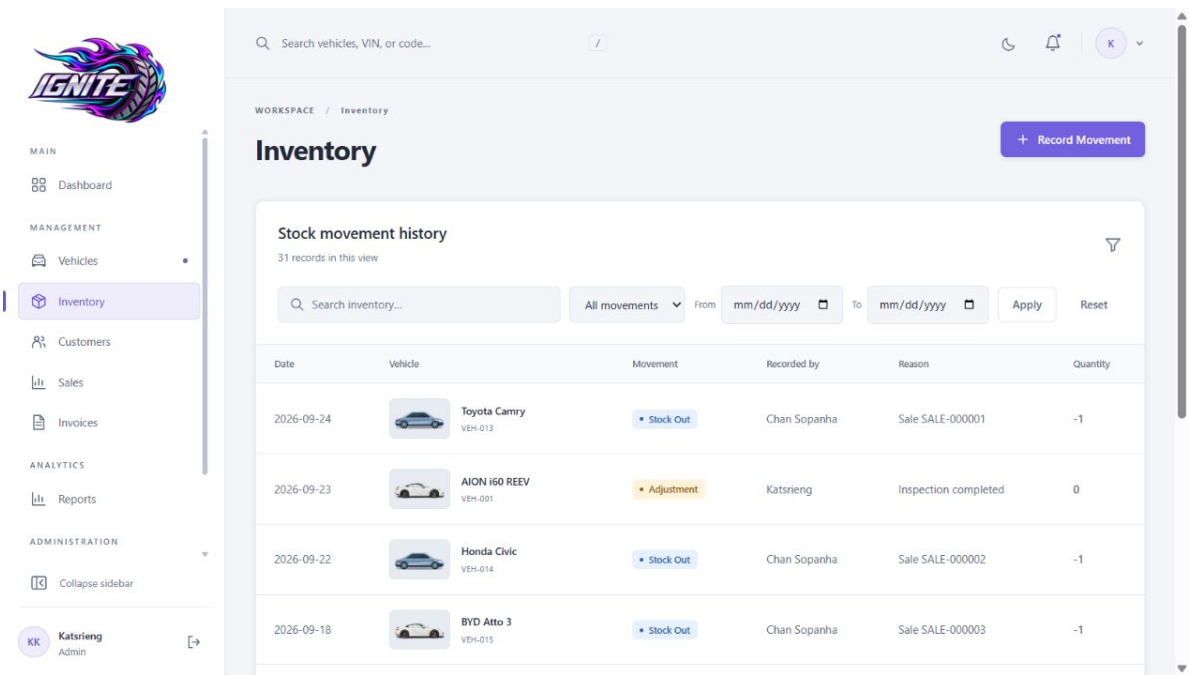


Figure 4.4. Inventory movement listing. Movement type, signed quantity and reason describe the history of individual vehicles.

The vehicle record is the source of current availability. Movement records explain later stock transitions. Staff use the listing to locate a record, open its details and perform actions permitted for their role.

4 Customer and sales screens

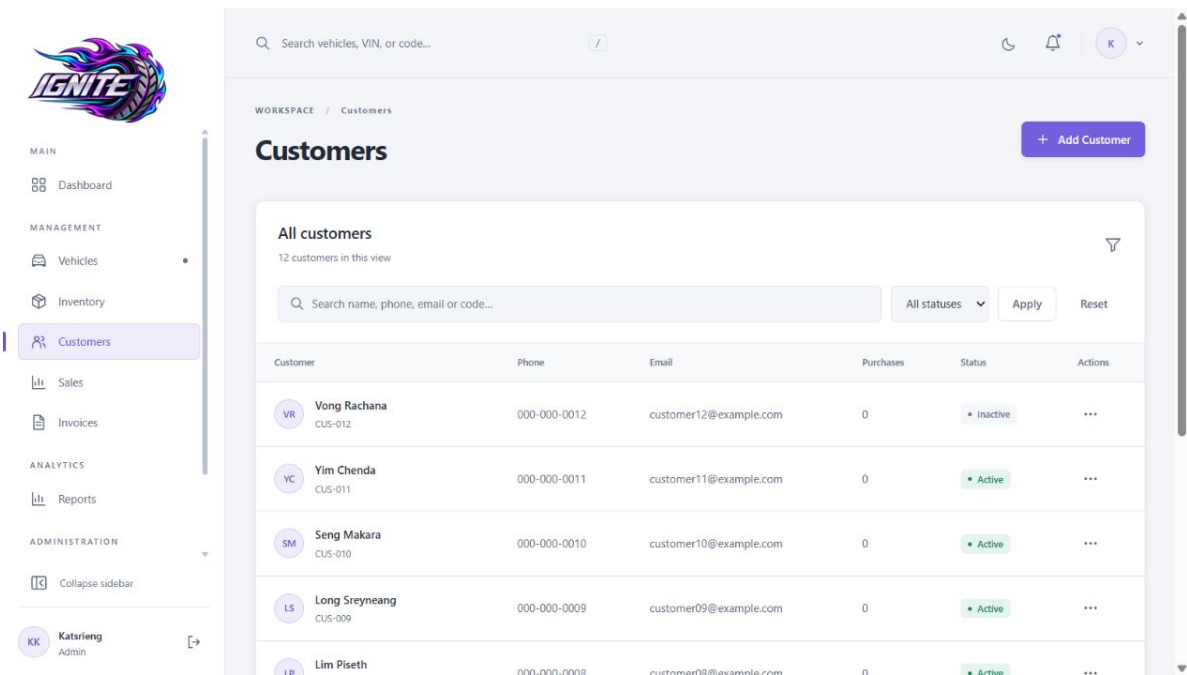


Figure 4.5. Customer listing populated only with the project's fictional seeded contacts.

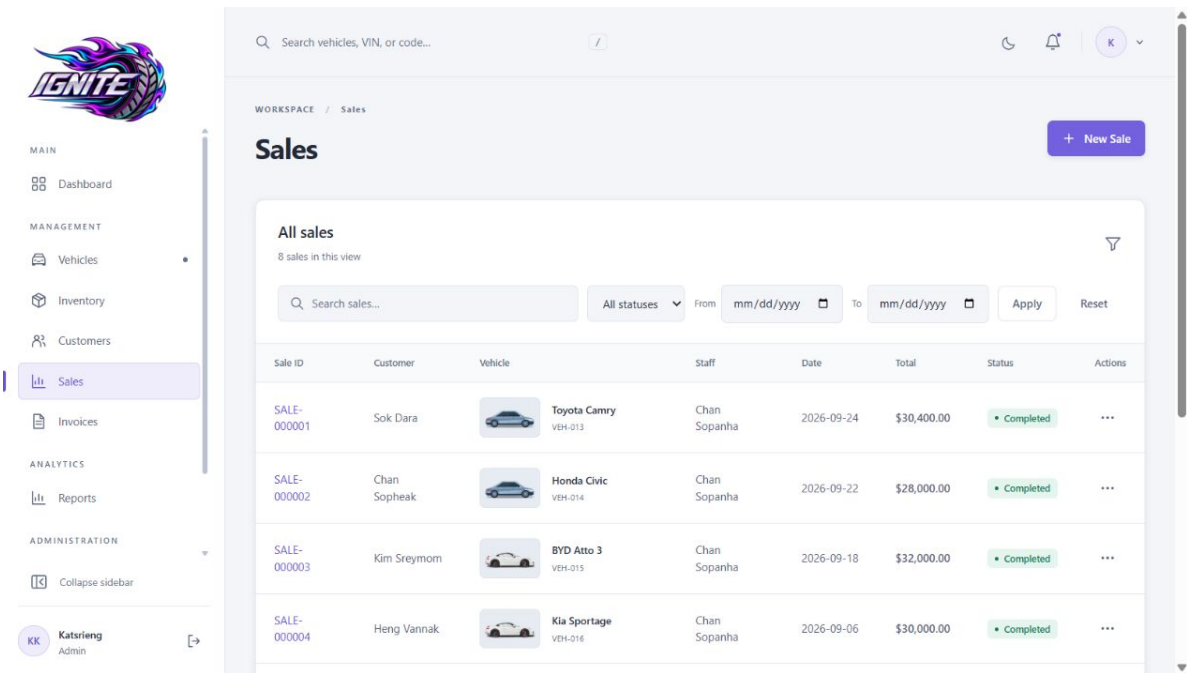


Figure 4.6. Sales listing showing completed transactions and cleaned seed sale references.

Customer information is linked to sales by customer_id. The New Sale workflow obtains the price from the selected vehicle and sends completion to the server, where availability and amounts are validated again. This server-side check remains necessary even when the interface only presents available vehicles.

4 Invoice and reporting screens

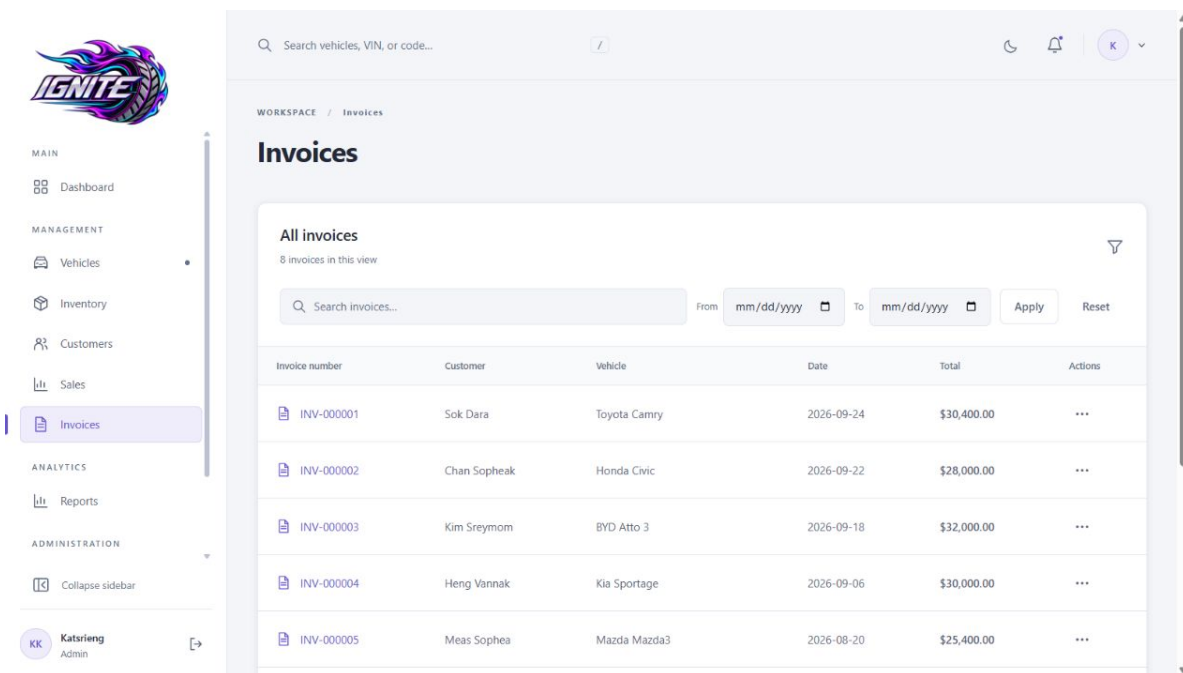


Figure 4.7. Invoice listing with unique invoice numbers, linked vehicle/customer information and date filters.

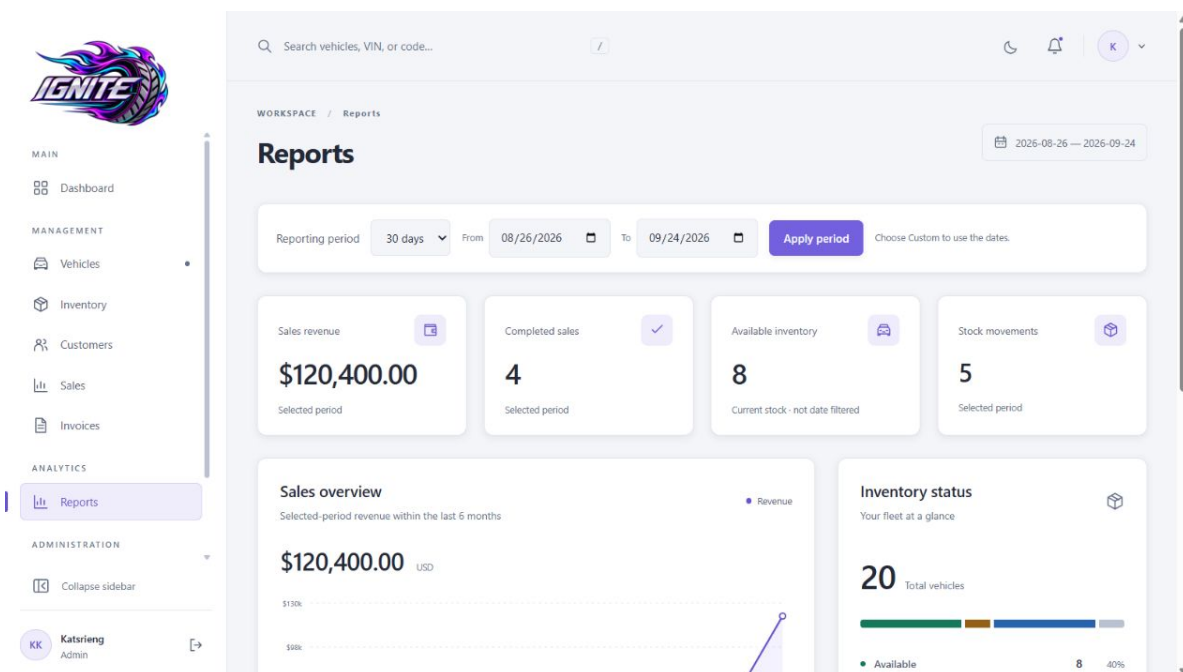


Figure 4.8. Reports page combining current inventory information and sales analytics. Period selection applies to supported sales and movement measures.

Reports use database aggregation rather than hard-coded totals. Inventory is a current snapshot; the revenue chart covers the last six calendar months within the selected period. These screens demonstrate seeded behavior, not actual dealership revenue.

4 Invoice detail



IGNITE
Vehicle Inventory & Sales Management

INVOICE
INV-000001

BILL TO
Sok Dara
Sample address 1, Phnom Penh, Cambodia
customer01@example.com
000-000-0001

Issue date 2026-09-24
Sale reference SALE-000001
Currency USD

Description	Quantity	Amount
Toyota Camry - 2024 VIN: 9ZZ1K8R2000000013 Vehicle code: VEH-013	1	\$30,900.00
Subtotal		\$30,900.00
Discount		-\$500.00
Total		\$30,400.00

Thank you for choosing IGNITE.
Enjoy the road ahead.
Development invoice - Not a tax invoice. Taxes and dealership legal details must be configured before production use.

Figure 4.9. Invoice INV-000001 linked to SALE-000001 and vehicle VEH-013. The fictional customer and vehicle data come from the central seed.

The example displays a USD 30,900.00 subtotal and USD 500.00 discount, producing a USD 30,400.00 total. This connects the sale amount, invoice total and vehicle identity in one view. The page includes a print action and explicitly identifies the document as a development invoice.

The invoice currently joins customer and vehicle information from their records. It does not yet preserve immutable snapshots of every historical address or legal identity field. Consequently, changing source information may affect how an older invoice is displayed. Snapshotting and configured tax/legal details are listed as future work, rather than claimed as completed features.

5 Testing and Results

5.1 Test method and environment

The existing unittest suite was executed against the reviewed project on 24 September 2026. Database tests created uniquely named temporary databases, seeded them and removed only those owned test databases. A separate temporary seeded database supplied the screenshots. The working project database was not reset or reseeded for this report.

The test helper requires MySQL 8 and port 3307. The local MySQL 8.0.46 service was available on port 3306, so a temporary loopback forward exposed it at 3307 without changing application configuration. `RUN_MYSQL_TESTS=1` enabled the database tests. The test account needed permission to create and drop its isolated databases.

Measure	Observed result
Tests executed	51
Passed	51
Failures / errors / skipped	0 / 0 / 0
unittest-reported duration	103.855 seconds
Reviewed commit	a2f79b8f88be18c6748d17bbbe20bd034aa55bd6
Source test modules	test_application, test_database_setup, test_identifier_migration, test_mysql_integration, test_vehicle_photos

```
RUN_MYSQL_TESTS=1
python -m unittest discover -s tests -v
```

```
-----
Ran 51 tests in 103.855s
```

```
OK
```

Listing 5.1. Command intent and final runner output. `DB_HOST` and `DB_PORT` were set for the local test forward. The evidence log is `output/report/evidence/test-results.txt` in the workspace. The suite result is functional regression evidence, not a security audit or performance benchmark.

5.2 Test distribution

Module	Count	Focus
Application	11	Pages, forms, permissions, transactions and inventory
Database setup	5	Entry point, schema order, seed definitions and safe names
Identifier migration	4	Clean identifiers, preview/apply, collisions and idempotence
MySQL integration	11	Persistence, rollback, money, access and seed behavior
Vehicle photos	20	Validation, lifecycle, path safety and failure cleanup

5 Representative test cases

Case	Input or action	Expected result	Result
TC01	Valid login; invalid or absent CSRF token	Valid login succeeds; invalid protected submission rejected.	Pass
TC02	Sales Staff requests restricted administrative action	Server refuses action regardless of navigation visibility.	Pass
TC03	Complete sale for available vehicle	One sale, one invoice, one stock-out; vehicle SOLD.	Pass
TC04	Concurrent attempts to sell the same vehicle	At most one successful sale; duplicate rejected.	Pass
TC05	Force invoice write failure during completion	Sale and stock changes roll back.	Pass
TC06	Negative/excessive discount or inactive participant	Reject completion without partial record changes.	Pass
TC07	Decimal amounts through sale, invoice and report	Exact expected two-decimal totals.	Pass
TC08	STOCK_IN, STOCK_OUT and ADJUSTMENT transitions	Legal states change correctly; invalid transitions rejected.	Pass
TC09	SQL-injection-shaped search and duplicate unique values	Input treated as data; constraints remain effective.	Pass
TC10	Construct fresh app instance	Previously committed data remains available.	Pass
TC11	Deactivate last administrator or change own access	Protected access change rejected.	Pass
TC12	Disguised, oversized or unsupported image upload	Reject upload while preserving record and old photo.	Pass
TC13	Replace/remove photo; simulate save/commit failure	Safe cleanup ordering; no incorrect deletion of old/shared file.	Pass
TC14	Preview/apply legacy identifier migration twice	Only planned fields change; repeat is a no-op.	Pass
TC15	Identifier collision or inconsistent seed history	Abort safely rather than overwrite unrelated records.	Pass
TC16	Database outage	Friendly 503 response without credentials or raw SQL.	Pass

These are grouped report cases mapped to the existing test functions, not an assertion that the suite contains only sixteen tests. The full run contains 51 tests. Browser screenshots provide additional display evidence but do not substitute for the transactional assertions.

5 Corrections and validation

5.3 Implemented corrections and regression protection

Issue addressed	Implemented correction	Evidence
Risk of double-selling or partial sale persistence	Row locks, unique vehicle sale constraint and one transaction.	Concurrency and invoice-failure rollback tests.
Combined modules difficult to navigate	Split domain, repository, service and feature route packages.	Current structure and refactor-verification.md.
Legacy sample identifiers visible in business records	Clean seed definitions and guarded identifier migration.	Four identifier tests and current screenshots.
Uploaded file deletion before a successful save	Post-commit reference-aware cleanup.	Replacement, shared-file and commit-failure tests.
Access changes could remove required administration	Last-admin, self-access and protected-grant checks.	User/role integration tests.
Raw database failures unsuitable for the interface	Application exceptions and friendly outage response.	Database outage test.

This table identifies corrections present in the code and supported by tests or repository documentation. It does not invent a historical bug-discovery date, author or failing-before test run. Historical verification notes describe earlier revisions; the fresh result in Section 5.1 applies to the reviewed commit.

5.4 Validation and security boundaries

The implemented safeguards include password hashing, active-account checks, server-side permission checks, CSRF validation, parameterized query values, bounded fields, database constraints and safe image decoding. Session cookies are HttpOnly and SameSite=Lax; HTTPS-only cookies are configurable for an HTTPS deployment. Local development is bound to loopback with debug disabled.

These controls do not establish production readiness. Login rate limiting, an appropriately restricted deployment database account, a production WSGI server, HTTPS and tested backup/restore procedures remain deployment tasks. Uploaded photos are static resources and are not individually authorization-protected. No penetration test, external security audit, load test or accessibility audit was performed for this report.

5.5 Demonstration results

The fresh sample dashboard showed 20 vehicles: 8 AVAILABLE, 2 RESERVED, 8 SOLD and 2 INACTIVE. The seed produced 12 customers, 8 completed sales, 8 invoices and 31 movements. The eight seeded sale totals sum to USD 256,000. The invoice example independently shows 30,900 minus 500 equals 30,400. These observations demonstrate coherent seeded relationships; they are not evidence of business adoption.

6 Conclusion

6.1 Achievements

IGNITE implements the proposal's core objective: a centralized, browser-based vehicle inventory and sales system using Python, Flask, MySQL and an object-oriented structure. Users can maintain individual vehicles and customers, inspect movement history, complete sales, view invoices and consult reports according to their permissions. The sale workflow maintains the relationship between availability, sale, stock-out and invoice in one transaction.

The five research and development questions in the proposal can be answered at the implementation level. Centralized records make vehicle information available through one interface. Stock movements retain vehicle/operator/date/reason history. Row locks and uniqueness prevent duplicate sales in the tested scenario. Role permissions control server actions, and layered objects separate rules from persistence and presentation. The project does not yet quantify improvements in staff time or error rates in a real dealership.

6.2 Limitations

The application completes sales immediately and has no refund, cancellation, instalment or payment gateway workflow. A unique vehicle sale relationship also means buy-back and resale are not modelled. RESERVED is a vehicle state rather than a complete reservation subsystem. Inventory is a current snapshot, not a historical valuation or full accounting ledger.

Invoices need legal configuration, tax handling and immutable identity snapshots before production use. VIN-like sample values are fictional and do not demonstrate registration or checksum verification. New sales use SAL- while cleaned seed sales use SALE-. The proposed Assistant role remains unimplemented. Photo storage requires backup alongside the database and can leave orphan files after a process crash.

6.3 Lessons learned

The project illustrates why transaction ownership belongs around a business operation rather than around separate table writes. It also shows that the user interface cannot replace server-side validation: a vehicle may become unavailable after a form is opened. Separating dataclasses, services and repositories makes these rules easier to locate, while preserving dictionary mappings avoids unnecessary template changes.

Tests for rollback, duplicate requests and file cleanup are particularly useful because failures occur between steps that may appear successful individually. Seed repeatability and guarded migration are also part of reliability: development tooling must not silently replace existing records or edited access grants.

6.4 Future work

The next priorities are invoice snapshots and legal configuration, consistent generated identifiers, deployment hardening and a backup/restore rehearsal. Later iterations can implement a defined cancellation/refund state machine, an Assistant role with agreed permissions and formal user acceptance testing. Multi-branch support, external services and advanced analytics should follow only after their additional consistency and privacy requirements are designed.

Appendix A SQL access tables

Source: database/schema.sql at a2f79b8. Statements are reproduced in original order; long lines are wrapped for printing. Use database/setup.py for environment-aware initialization.

```
-- MySQL 8.0.16+ (verified on MySQL 8.0.46).
-- setup.py safely substitutes this database identifier from DB_NAME in .env.
CREATE DATABASE IF NOT EXISTS `VehicleSalesDB` CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
USE `VehicleSalesDB`;

CREATE TABLE IF NOT EXISTS roles (
  role_id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  role_name VARCHAR(80) NOT NULL UNIQUE,
  description VARCHAR(250) NOT NULL DEFAULT '',
  is_active BOOLEAN NOT NULL DEFAULT TRUE,
  created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
) ENGINE=InnoDB;

CREATE TABLE IF NOT EXISTS permissions (
  permission_id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  permission_name VARCHAR(100) NOT NULL UNIQUE,
  module VARCHAR(50) NOT NULL,
  action VARCHAR(50) NOT NULL,
  description VARCHAR(250) NOT NULL DEFAULT '',
  is_active BOOLEAN NOT NULL DEFAULT TRUE,
  created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
) ENGINE=InnoDB;

CREATE TABLE IF NOT EXISTS role_permissions (
  role_id INT UNSIGNED NOT NULL,
  permission_id INT UNSIGNED NOT NULL,
  created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY (role_id, permission_id),
  FOREIGN KEY (role_id) REFERENCES roles(role_id),
  FOREIGN KEY (permission_id) REFERENCES permissions(permission_id)
) ENGINE=InnoDB;

CREATE TABLE IF NOT EXISTS users (
  user_id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  username VARCHAR(80) NOT NULL UNIQUE,
  password_hash VARCHAR(255) NOT NULL,
  full_name VARCHAR(250) NOT NULL,
  email VARCHAR(250) NULL UNIQUE,
  is_active BOOLEAN NOT NULL DEFAULT TRUE,
  created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
) ENGINE=InnoDB;

CREATE TABLE IF NOT EXISTS user_roles (
  user_id INT UNSIGNED NOT NULL,
  role_id INT UNSIGNED NOT NULL,
  assigned_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY (user_id, role_id),
  FOREIGN KEY (user_id) REFERENCES users(user_id),
  FOREIGN KEY (role_id) REFERENCES roles(role_id)
) ENGINE=InnoDB;
```

Appendix A SQL customers and vehicles

Source: database/schema.sql at a2f79b8. Statements are reproduced in original order; long lines are wrapped for printing. Use database/setup.py for environment-aware initialization.

```
CREATE TABLE IF NOT EXISTS customers (
  customer_id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  customer_code VARCHAR(80) NOT NULL UNIQUE,
  full_name VARCHAR(250) NOT NULL,
  phone VARCHAR(80) NOT NULL,
  email VARCHAR(250) NOT NULL,
  address VARCHAR(250) NOT NULL DEFAULT '',
  is_active BOOLEAN NOT NULL DEFAULT TRUE,
  created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
) ENGINE=InnoDB;

CREATE TABLE IF NOT EXISTS vehicles (
  vehicle_id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  vehicle_code VARCHAR(80) NOT NULL UNIQUE,
  vin VARCHAR(17) NULL UNIQUE,
  plate_number VARCHAR(80) NULL UNIQUE,
  brand VARCHAR(100) NOT NULL,
  model VARCHAR(150) NOT NULL,
  vehicle_year SMALLINT UNSIGNED NOT NULL,
  color VARCHAR(100) NOT NULL,
  purchase_price DECIMAL(14,2) NOT NULL,
  selling_price DECIMAL(14,2) NOT NULL,
  status VARCHAR(20) NOT NULL DEFAULT 'AVAILABLE',
  image_path VARCHAR(250) NOT NULL DEFAULT 'sedan.svg',
  mileage INT UNSIGNED NOT NULL DEFAULT 0,
  fuel VARCHAR(80) NOT NULL DEFAULT 'Not specified',
  created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  CONSTRAINT vehicle_prices CHECK (purchase_price >= 0 AND selling_price >= 0),
  CONSTRAINT vehicle_status CHECK (status IN ('AVAILABLE','RESERVED','SOLD','INACTIVE')),
  CONSTRAINT vehicle_vin CHECK (vin IS NULL OR CHAR_LENGTH(vin) = 17),
  INDEX vehicle_filters (status, brand, vehicle_year)
) ENGINE=InnoDB;
```

Appendix A SQL sales and inventory

Source: database/schema.sql at a2f79b8. Statements are reproduced in original order; long lines are wrapped for printing. Use database/setup.py for environment-aware initialization.

```
CREATE TABLE IF NOT EXISTS stock_movements (
  movement_id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  vehicle_id INT UNSIGNED NOT NULL,
  user_id INT UNSIGNED NOT NULL,
  movement_type VARCHAR(20) NOT NULL,
  movement_date DATE NOT NULL,
  reason VARCHAR(250) NOT NULL,
  quantity SMALLINT NOT NULL,
  created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (vehicle_id) REFERENCES vehicles(vehicle_id),
  FOREIGN KEY (user_id) REFERENCES users(user_id),
  CONSTRAINT movement_delta CHECK ((movement_type='STOCK_IN' AND quantity=1) OR
(movement_type='STOCK_OUT' AND quantity=-1) OR (movement_type='ADJUSTMENT' AND quantity=0)),
  INDEX movement_dates (movement_date, movement_type)
) ENGINE=InnoDB;

CREATE TABLE IF NOT EXISTS sales (
  sale_id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  sale_code VARCHAR(80) NOT NULL UNIQUE,
  user_id INT UNSIGNED NOT NULL,
  customer_id INT UNSIGNED NOT NULL,
  vehicle_id INT UNSIGNED NOT NULL UNIQUE,
  sale_date DATE NOT NULL,
  sale_price DECIMAL(14,2) NOT NULL,
  discount_amount DECIMAL(14,2) NOT NULL DEFAULT 0,
  total_amount DECIMAL(14,2) NOT NULL,
  status VARCHAR(20) NOT NULL DEFAULT 'COMPLETED',
  created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  FOREIGN KEY (user_id) REFERENCES users(user_id),
  FOREIGN KEY (customer_id) REFERENCES customers(customer_id),
  FOREIGN KEY (vehicle_id) REFERENCES vehicles(vehicle_id),
  CONSTRAINT sale_amounts CHECK (sale_price >= 0 AND discount_amount >= 0 AND discount_amount <=
sale_price AND total_amount = sale_price-discount_amount),
  CONSTRAINT sale_status CHECK (status IN ('PENDING','COMPLETED','CANCELLED')),
  INDEX sales_dates (status, sale_date)
) ENGINE=InnoDB;

CREATE TABLE IF NOT EXISTS invoices (
  invoice_id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  invoice_number VARCHAR(80) NOT NULL UNIQUE,
  sale_id INT UNSIGNED NOT NULL UNIQUE,
  issue_date DATE NOT NULL,
  total_amount DECIMAL(14,2) NOT NULL,
  created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (sale_id) REFERENCES sales(sale_id),
  CONSTRAINT invoice_amount CHECK (total_amount >= 0),
  INDEX invoice_dates (issue_date)
) ENGINE=InnoDB;
```

Appendix B User manual

B.1 Start a local installation

Use Python and MySQL 8.0.16 or later, then open a terminal in `vehicle_sales_system`. Create a virtual environment and install `requirements.txt`. For a fresh clone only, copy `.env.example` to `.env` and supply the local database settings and a random `SECRET_KEY`. Do not overwrite an existing `.env` or copy its secrets into Git.

```
python -m venv .venv
.\.venv\Scripts\python.exe -m pip install -r requirements.txt
# Fresh clone only; keep an existing .env.
Copy-Item .env.example .env
# Edit .env with this installation's MySQL settings and SECRET_KEY.
.\.venv\Scripts\python.exe database/setup.py
.\.venv\Scripts\python.exe app.py
```

Open `http://127.0.0.1:5000` in the browser and keep the terminal running. The actual local MySQL port used during this review was 3306; the test harness separately requires 3307. Check the installed server and `.env` rather than assuming the README's example port matches every machine. Normal startup does not reset records.

B.2 Daily workflows

Task	Steps
Sign in	Enter an existing account's username and password. The sidebar displays permitted modules.
Maintain a vehicle	Open Vehicles, choose Add Vehicle, complete identity and price fields, optionally attach a photo, and save. Open a row to inspect or edit it.
Track stock	Open Inventory and add a movement. Select a vehicle, movement and reason. Observe the allowed transitions in Section 3.12.
Maintain customers	Open Customers, add or edit contact details, then open the record to inspect related history.
Complete a sale	Open Sales and New Sale. Select an active customer and available vehicle, review the discount and complete. Inspect the generated invoice.
Print an invoice	Open Invoices, choose its number and use Print Invoice. The current document is not a legal tax invoice.
Review performance	Open Dashboard or Reports. Apply the required date period and distinguish current inventory from dated sales measures.
Manage access	As Admin, use Users and Roles & Permissions. Keep at least one active administrator.
End a session	Use Log out, especially on a shared computer.

Appendix B Maintenance guidance

B.3 Recover from common problems

Symptom	Check or action
Browser cannot connect	Confirm app.py is running, read its startup error and open the displayed loopback address.
Database unavailable message	Confirm MySQL is running and .env host, port, database and account are correct. Do not publish the password in a screenshot.
Permission denied	Use an account whose active role has the required permission; do not bypass the server check.
Vehicle unavailable during sale	Refresh availability. Another valid transaction may have completed first.
Photo rejected	Use a valid, non-animated JPEG, PNG or WebP within the configured size and pixel limits.
Duplicate code or VIN	Choose the correct existing record or enter a truly distinct value; do not bypass uniqueness.
Test database cases skipped	Set RUN_MYSQL_TESTS=1 only in the documented isolated MySQL test environment.

B.4 Safe data handling

Back up both MySQL and app/static/uploads/vehicles, because a database-only backup does not contain the image files. Keep real customer data and credentials out of screenshots used for teaching or submission. This report's application screenshots use fictional seed contacts in a temporary database.

database/migrate_identifiers.py previews recognized legacy sample labels. Its --apply option applies guarded changes and writes backup evidence. Review the preview and maintain a backup before using a migration on a working database. Code changes retain numeric relationships; a sale code is a label, not the foreign key used by its invoice.

Do not use reset/reseed commands against a database containing genuine user-created records. Repeating normal setup is designed to preserve existing records and grants, while inconsistent sample-sale history causes a conflict instead of silent repair. Development reset commands are outside the routine user workflow.

B.5 Reproduce the report checks

At the reviewed commit, install the project requirements, provide an isolated MySQL 8 test environment on port 3307, set RUN_MYSQL_TESTS=1 and run `python -m unittest discover -s tests -v`. The suite owns fresh uniquely named databases and drops those only. A skipped suite is not evidence that database behavior passed.

For presentation screenshots, initialize a separate sample database and pass it to `create_app`. Capture login, dashboard, vehicle, inventory, customer, sales, invoice and report pages. Do not switch the working dealership database merely to obtain cleaner screenshots. The temporary report server in this review used port 5002.

Appendix C Git evidence

C.1 Reviewed version

Item	Observed value
Repository	Katsrieng/UTE_Advanced_OOP_Final_Project
Branch	refactor/clean-structure
HEAD	a2f79b8f88be18c6748d17bbbe20bd034aa55bd6
Tracking branch	origin/refactor/clean-structure
Local status at report inspection	Clean; no ahead/behind difference shown against the locally stored tracking reference.
Review boundary	No new remote fetch, push or merge was performed for report preparation. This is local Git evidence, not a claim about later remote changes.

```
git status -sb
## refactor/clean-structure...origin/refactor/clean-structure
```

```
git log -1 --oneline
a2f79b8 refactor project structure and update vehicle sales system
```

C.2 Selected development history

Date	Commit and Git author	Recorded subject
24 Sep 2026	a2f79b8 ChansokpanhaSour	Refactor project structure and update vehicle sales system
24 Sep 2026	b410e28 Katsrieng	Update IGNITE development passwords
23 Sep 2026	ae2c662 Katsrieng	Merge latest main and keep README
23 Sep 2026	4c45cf4 Katsrieng	Integrate MySQL persistence and IGNITE branding with updated users
23 Sep 2026	9827586 Katsrieng	Implement vehicle photo CRUD with validated uploads and safe cleanup

The selected entries are read from git log. Git author strings identify commit metadata; they do not prove every team member's individual contribution or working hours. Earlier history contains legacy project wording, which is retained as historical evidence rather than rewritten. The repository URL is https://github.com/Katsrieng/UTE_Advanced_OOP_Final_Project.

Appendix D Team log

D.1 Team membership

The proposal lists Lay Katsrieng, Sour Chansokpanha and Heang Sovannara. It does not supply a completed individual activity log, student IDs or verified working hours. The available Git history provides limited commit evidence, but it is not sufficient to reconstruct every member's work. Individual contributions are therefore left unverified rather than invented.

Team member	Confirmed work and evidence	Dates or hours
Lay Katsrieng	To be confirmed by the team.	Not supplied
Sour Chansokpanha	To be confirmed by the team.	Not supplied
Heang Sovannara	To be confirmed by the team.	Not supplied

D.2 Evidence-backed project activity log

Date	Activity visible in evidence	Source
23 Sep 2026	Vehicle photo CRUD and safe upload cleanup recorded.	Git commit 9827586 and photo tests.
23 Sep 2026	MySQL persistence and IGNITE branding integration recorded.	Git commit 4c45cf4 and integration documentation.
24 Sep 2026	Package refactor and system updates recorded.	Git commit a2f79b8 and docs/refactor-verification.md.
24 Sep 2026	Fresh 51-test run completed with no failures or skips.	Report test-results.txt.
24 Sep 2026	Current screens captured using isolated seeded data.	Figures 4.1 to 4.9.

D.3 Submission completion note

Before submission, the team should confirm each member's actual tasks and dates, add student IDs if required by the lecturer, and replace the unverified entries above with agreed evidence. This is the only intentionally incomplete administrative section; implementation descriptions and test outcomes in the report are based on inspected project evidence.

The proposal's written timeline presents ten weeks, while its appended presentation shows a four-week plan. Neither is treated as an actual completion log. Appendix C and the activity entries above use observed repository dates, avoiding an invented development schedule.

References

- [1] Lay Katsrieng, Sour Chansokpanha and Heang Sovannara. Vehicle Inventory and Sales Management System for SMEs. Advanced OOP with Python proposal, UTE, Class CSM1, Term 5, Year 2. User-supplied OOP_Proposal.pdf; reviewed 24 September 2026.
- [2] Lecturer-provided Final Report Structure. User-supplied screenshot dated 23 September 2026. Required chapters and appendices reproduced in the report guide.
- [3] Pallets Projects. Design Decisions in Flask. Flask 3.1 documentation. Accessed 24 September 2026. <https://flask.palletsprojects.com/en/stable/design/>
- [4] Oracle. MySQL 8.0 Reference Manual, Locking Reads. Accessed 24 September 2026. <https://dev.mysql.com/doc/refman/8.0/en/innodb-locking-reads.html>
- [5] Python Software Foundation. dataclasses - Data Classes. Python documentation. Accessed 24 September 2026. <https://docs.python.org/3/library/dataclasses.html>
- [6] Odoo. Use serial numbers to track products. Odoo 17.0 documentation. Accessed 24 September 2026. [Odoo documentation - serial number tracking](#)
- [7] IGNITE source repository. Commit a2f79b8f88be18c6748d17bbbe20bd034aa55bd6, branch refactor/clean-structure. Primary implementation evidence: app/models, app/services, app/repositories, app/routes, app/database.py and database/schema.sql. [GitHub project repository](#)
- [8] IGNITE project documentation. README.md, database/README.md and docs/refactor-verification.md at the reviewed commit. Earlier integration documents are historical records, not substitutes for current source inspection.
- [9] IGNITE regression evidence. Existing tests/test_*.py executed on 24 September 2026. Workspace log: output/report/evidence/test-results.txt. 51 passed, 0 failures, 0 errors, 0 skipped.
- [10] IGNITE browser evidence. Figures 4.1 to 4.9 captured from the reviewed source with an isolated seeded MySQL database on 24 September 2026. Sample values are not real dealership transactions.

Evidence interpretation

References [1] and [2] define the requested academic scope. References [3] to [6] support the technology review. References [7] to [10] support the implemented design, screenshots and test results. Report diagrams were drawn from the current classes and schema rather than copied from the proposal's earlier ERD.